

Python

Module 1: What is a Program, Memory & Languages

1.1 What is a Program?

A program is a collection of instructions that can be executed by a computer to perform a specific task. Every program is written in a programming language that the computer can understand — either directly or after translation.

**NOTE**

From the PDF: 'A program is a collection of instructions that can be executed by a computer to perform a specific task.'

1.2 Memory — How a Computer Stores Data

RAM (Random Access Memory)

RAM is a temporary memory directly connected to the microprocessor. All execution of programs happens on RAM. It is volatile — when power is cut, all data in RAM is lost.

- RAM stores instructions before they are given to the microprocessor
- Microprocessor is connected only to RAM, not directly to hard disk
- RAM is semiconductor technology — made of transistors

Hard Disk

Hard disk is permanent (non-volatile) magnetic storage. Programs stored here are called files. The process of transferring from RAM to hard disk is called saving, and loading is the reverse.

How Python Programs Execute

Python source code (HLL) → Python Interpreter → Machine Level Language → CPU executes

```
# Python source (.py file) on Hard Disk
# ↓ Load into RAM
# Python Interpreter reads line by line
# ↓ Converts each line to Machine Level Code
# CPU executes → Output
```

1.3 Levels of Programming Languages

Level	Language	Era	Example
Low Level	Machine Level (MLL)	1940s	0110110 = add
Low Level	Assembly Level (ALL)	1950s	ADD AX, BX
High Level	High Level (HLL)	1960s+	a = b + c

1.4 Compiler vs Interpreter

Feature	Compiler	Interpreter
Translation	Entire program at once	One line at a time
Speed	Faster execution	Slower execution
Error Detection	After full compilation	At each line
Examples	C, C++, Java	Python, Ruby
Output	Executable file created	No separate file created

NOTE

Python is an INTERPRETED language. The interpreter converts each line of Python to Machine Level Code and executes it immediately before moving to the next line.

1.5 Practice Questions — Programs, Memory & Languages

Q1. What is the difference between RAM and Hard Disk?

A1. RAM is temporary (volatile) memory directly connected to the CPU — programs execute here. Hard disk is permanent (non-volatile) storage — files are stored here. When power is off, RAM data is lost, but hard disk data is retained.

Q2. Is Python compiled or interpreted? Explain with the execution flow.

A2. Python is interpreted. Execution flow: Python source file (.py) is loaded → Python interpreter reads one line at a time → converts each line to Machine Level Code → CPU executes → moves to next line. No separate compiled executable is created.

Q3. What is the difference between a Compiler and an Interpreter?

A3. A compiler translates the ENTIRE program at once into Machine Level Code before execution begins. An interpreter translates and executes ONE LINE AT A TIME. Compilers produce faster-running programs; interpreters are easier for debugging since errors are shown line by line.

Module 2: Python Introduction, Features & Versions

2.1 What is Python?

Python is an interpreted, high-level, general-purpose programming language created by Guido van Rossum and first released in 1991. Its design philosophy emphasizes code readability with significant whitespace. It supports multiple programming paradigms: structured, object-oriented, and functional programming.

2.2 Python is Used For

- Web development (server-side) — Django, Flask
- Software development and scripting
- Data science and machine learning — NumPy, Pandas, TensorFlow
- Mathematics and scientific computing
- System scripting and automation
- Artificial intelligence and deep learning

2.3 Key Features of Python

Feature	Description
Easy to Learn	Simple syntax resembling English; readable code
Interpreted	Executes line by line — no compilation step needed
High Level	Programmer does not manage memory manually
Dynamically Typed	No need to declare variable types — Python infers them
Object Oriented	Supports classes, objects, inheritance, polymorphism
Free & Open Source	Free to download, use and distribute
Portable	Same Python code runs on Windows, macOS, Linux
Large Standard Library	Hundreds of built-in modules (math, os, sys, datetime...)
General Purpose	Used for web, AI, scripts, automation, data science

2.4 History & Versions

- Late 1980s — Guido van Rossum began design at CWI, Netherlands
- 1991 — Python 0.9.0 released (first public version)
- 2000 — Python 2.0 released (list comprehensions, garbage collection)
- 2008 — Python 3.0 released (major redesign; NOT backward compatible with Python 2)
- 2020 — Python 2 officially retired (no more support)
- Current — Python 3.x is the standard (3.11, 3.12...)

⚠ CAUTION

Python 2 and Python 3 are NOT fully compatible. Always use Python 3 for all new projects.

2.5 Python vs C vs Java — A Comparison

Example: Swap two numbers

```
# Python — simplest (3 lines)
a, b = 10, 20
a, b = b, a
print(a, b)

// Java — verbose (needs class, types, main method)
// C — needs temp variable manually
```

NOTE

Python programs are 3 to 5 times shorter than equivalent Java or C programs. This makes Python extremely productive for developers.

2.6 Interactive Mode vs Script Mode

Feature	Interactive Mode	Script Mode
How to use	Type commands at >>> prompt	Write .py file, run it
Saving	Cannot save code	Code saved in .py file
Best for	Quick testing, learning	Full programs, projects
Run command	python (or python3)	python filename.py

2.7 Practice Questions — Python Intro

Q1. Who created Python and in which year was it first released?**A1.** Python was created by Guido van Rossum. It was first released in 1991. Guido began designing it in the late 1980s at CWI (Centrum Wiskunde & Informatica) in the Netherlands as a successor to the ABC language. Fun fact: Python's name comes from the British comedy group 'Monty Python', which Guido enjoyed.**Q2.** Name five key features of Python.**A2.** (1) Easy to learn — simple English-like syntax. (2) Interpreted — executes line by line. (3) Dynamically typed — no need to declare variable types. (4) Object-Oriented — supports classes, inheritance, polymorphism. (5) Large standard library — hundreds of built-in modules for math, files, networking, etc.**Q3.** What is the difference between Interactive Mode and Script Mode?**A3.** Interactive mode: code is typed at the >>> prompt and executed immediately; cannot be saved. Best for quick testing. Script mode: code is written in a .py file and run using 'python filename.py'; code can be saved, edited, and rerun. Best for full programs and projects.

Module 3: Data Types, Variables & Operators

3.1 Variables

A variable is a named container that stores a value in memory. In Python, variables are created when you assign a value — no type declaration needed. Python infers the type automatically.

```
name = 'Ravi'      # str
age = 25           # int
salary = 45000.50  # float
is_active = True   # bool

# Multiple assignment
x = y = z = 0
a, b, c = 10, 20, 30

# Check type
print(type(age))    # <class 'int'>
print(id(age))      # Memory address
```

Variable Scope

- Global Variable — declared outside all functions; accessible everywhere in the file
- Local Variable — declared inside a function; accessible only within that function

```
count = 10          # Global variable

def show():
    msg = 'Hello'    # Local variable
    print(count)     # Can access global
    print(msg)

# print(msg) → Error! msg is local to show()
```

3.2 Python Data Types

Data Type	Keyword	Example	Notes
Integer	int	age = 25	Whole numbers, no decimal
Float	float	price = 9.99	Decimal / floating point numbers
Complex	complex	z = 3+4j	Real + imaginary part
String	str	name = 'Ravi'	Text, enclosed in quotes
Boolean	bool	flag = True	Only True or False values
List	list	[1,2,3]	Ordered, mutable, allows duplicates
Tuple	tuple	(1,2,3)	Ordered, immutable
Set	set	{1,2,3}	Unordered, mutable, no duplicates
Dictionary	dict	{'a':1}	Key-value pairs, mutable

Mutable vs Immutable Data Types

Category	Types	Meaning
Immutable	int, float, str, bool, tuple	Value cannot be changed in-place; a new object is created on change
Mutable	list, dict, set	Value can be changed in-place; same object is modified

3.3 Typecasting

Converting one data type to another is called typecasting. Python provides built-in functions for this.

```
int('42')          # str → int    → 42
float(10)           # int → float  → 10.0
str(25)             # int → str    → '25'
bool(0)             # int → bool   → False
list((1,2,3))       # tuple→list  → [1,2,3]
tuple([1,2,3])      # list→tuple  → (1,2,3)
complex(3,4)        # → 3+4j
```

⚠ CAUTION

Converting float to int truncates the decimal part: `int(9.9) → 9`. The 0.9 is permanently lost!

3.4 Operators

Arithmetic Operators

Operator	Name	Example	Result
+	Addition	10 + 3	13
-	Subtraction	10 - 3	7
*	Multiplication	10 * 3	30
/	Division (float)	10 / 3	3.333...
//	Floor Division	10 // 3	3 (integer result)
%	Modulus (remainder)	10 % 3	1
**	Exponentiation	2 ** 8	256

Comparison Operators

```
==  !=  >  <  >=  <=  # All return True or False
```

Logical Operators

Operator	Meaning	Example
and	True if BOTH conditions are True	(5>3) and (10>7) → True
or	True if AT LEAST ONE is True	(5>3) or (1>7) → True
not	Reverses the boolean result	not(5>3) → False

Assignment Operators

```
x = 10          # Assign
x += 5          # x = x + 5   → 15
x -= 3          # x = x - 3   → 12
x *= 2          # x = x * 2   → 24
x /= 5          # x = x / 5   → 4
x **= 3         # x = x ** 3  → 64
```

Identity & Membership Operators

```
# Identity - checks if two variables point to the SAME object
a = [1,2,3]; b = a
print(a is b)          # True - same object
print(a is not b)      # False

# Membership - checks if value is IN a sequence
fruits = ['apple','mango']
print('mango' in fruits) # True
print('grape' not in fruits) # True
```

3.5 Practice Questions — Data Types & Operators

Q1. What is the output of: `print(type(3+4j))`?

A1. `<class 'complex'>` — 3+4j is a complex number (real part=3, imaginary part=4).

Q2. What is the difference between `/` and `//` in Python?

A2. `/` (division) always returns a float: `10/3 → 3.3333`. `//` (floor division) returns the integer quotient, discarding the remainder: `10//3 → 3`. Use `//` when you need a whole number result.

Q3. What is typecasting? Convert the string '42' to an integer and add 8.

A3. Typecasting is converting one data type to another using built-in functions.

```
num = int('42') + 8
print(num)      # Output: 50
```

Q4. Which data types are mutable and which are immutable in Python?

A4. IMMUTABLE (cannot be changed in-place): int, float, str, bool, tuple. MUTABLE (can be changed in-place): list, dict, set. When you 'modify' an immutable type, Python creates a NEW object; the original is unchanged. Mutable types modify the same object in memory.

Q5. What is the difference between `'=='` and `'is'` operators?

A5. == checks if two variables have the SAME VALUE. 'is' checks if two variables point to the SAME object in memory (same identity/address). Example: a=[1,2]; b=[1,2]; a==b is True (same value) but a is b is False (different objects).

Module 4: Control Structures — if, Loops

4.1 Conditional Statements

if Statement

A condition is checked. If True, the block under 'if' executes. If False, it is skipped.

SYNTAX

if condition: statement(s)

```
marks = 85
if marks >= 60:
    print('Passed!')    # Executes only if marks >= 60
```

if-else Statement

```
num = 7
if num % 2 == 0:
    print('Even')
else:
    print('Odd')        # Output: Odd
```

if-elif-else Statement

Used when there are multiple conditions to check. 'elif' is short for 'else if'.

```
marks = 75
if marks >= 90:
    print('Grade: A+')
elif marks >= 80:
    print('Grade: A')
elif marks >= 70:
    print('Grade: B')
elif marks >= 60:
    print('Grade: C')
else:
    print('Grade: F')    # Output: Grade: B
```

NOTE

Python does NOT have a built-in switch/case statement (before Python 3.10). Use if-elif-else chains. Python 3.10+ introduced 'match-case' as an alternative.

False Values in Python

These values evaluate to False in a condition:

```
False, None, 0, 0.0, '', (), [], {}
```

4.2 Looping Statements

while Loop

Executes a block AS LONG AS the condition is True. Use when you don't know how many iterations are needed.

SYNTAX

while condition: body of loop

```
# Print numbers 1 to 5
i = 1
while i <= 5:
    print(i)
    i += 1          # Output: 1 2 3 4 5

# Bank withdrawal - loop while balance is sufficient
balance = 10000
while balance >= 2000:
    balance -= 2000
    print('Withdrew 2000, Balance:', balance)
```

for Loop

Iterates over a sequence (list, tuple, dict, set, string, range). Use when you know how many iterations are needed.

SYNTAX

for variable in iterable: body of loop

```
# Iterate over a list
fruits = ['apple', 'mango', 'banana']
for fruit in fruits:
    print(fruit)

# Iterate using range()
for i in range(1, 6):    # 1,2,3,4,5
    print(i)

# Iterate over string characters
for char in 'Python':
    print(char)
```

range() Function

SYNTAX

range(start, stop, step) # stop is EXCLUDED

```
range(5)          # 0,1,2,3,4
range(1, 6)       # 1,2,3,4,5
range(0, 10, 2)    # 0,2,4,6,8 (step=2)
range(10, 0, -1)   # 10,9,8,7,6,5,4,3,2,1 (countdown)
list(range(5))     # [0,1,2,3,4]
```

Loop Control Statements

```
# break - exits the loop immediately
for i in range(10):
```

```

    if i == 5:
        break
    print(i)          # Output: 0 1 2 3 4

# continue - skips current iteration, continues loop
for i in range(6):
    if i == 3:
        continue
    print(i)          # Output: 0 1 2 4 5

# pass - placeholder (does nothing)
for i in range(5):
    pass              # No operation

```

Nested Loops

```

# Multiplication table for 2
for i in range(1, 11):
    print(f'2 x {i} = {2*i}')

# Nested: print a 3x3 star pattern
for row in range(3):
    for col in range(3):
        print('*', end=' ')
    print()           # New line after each row

```

4.3 When to Use for vs while

	for loop	while loop
Use When	Exact number of iterations known	Number of iterations unknown
Driven by	Iterable or range()	Condition
Example	Iterate 5 times	Withdraw until balance < 2000

4.4 Practice Questions — Control Structures

Q1. Write a Python program to check if a given number is positive, negative, or zero.

A1. Use if-elif-else:

```

num = int(input('Enter number: '))
if num > 0:
    print('Positive')
elif num < 0:
    print('Negative')
else:
    print('Zero')

```

Q2. Print all odd numbers from 1 to 20 using a for loop.

A2. Use range with step 2, starting from 1:

```

for i in range(1, 21, 2):
    print(i)

```

Q3. Write a while loop to find the sum of digits of a number (e.g., 1234 → 10).

A3. Keep extracting the last digit using modulus:

```
num = 1234
total = 0
while num > 0:
    total += num % 10
    num //= 10
print('Sum of digits:', total) # 10
```

Q4. Print the multiplication table for any number entered by the user.

A4. Combine input() and a for loop:

```
n = int(input('Enter number: '))
for i in range(1, 11):
    print(f'{n} x {i} = {n*i}')
```

Module 5: Functions & Recursion

5.1 What is a Function?

A function is a named block of reusable code that performs a specific task. Functions take input (parameters), process it, and optionally return output.

- Reduces code duplication — write once, call many times
- Improves readability and maintainability
- Enables modular programming — divide complex programs into smaller parts

5.2 Defining & Calling Functions

SYNTAX `def function_name(parameters):` body return value

```
# 1. No input, no return
def greet():
    print('Hello, World!')
greet()

# 2. With input, no return
def show_name(name):
    print(f'Hello, {name}!')
show_name('Ravi')

# 3. No input, with return
def get_pi():
    return 3.14159
value = get_pi()

# 4. With input, with return
def add(a, b):
```

```

    return a + b
result = add(10, 20)    # 30

```

5.3 Types of Arguments

```

# Positional arguments – matched by position
def power(base, exp):
    return base ** exp
power(2, 8)    # base=2, exp=8

# Keyword arguments – matched by name
power(exp=3, base=5)    # 125

# Default arguments – used if not provided
def greet(name, msg='Hello'):
    print(f'{msg}, {name}!')
greet('Ravi')    # Hello, Ravi!
greet('Priya', 'Good day') # Good day, Priya!

# Variable-length arguments (*args)
def total(*nums):
    return sum(nums)
total(1,2,3,4,5)    # 15

# Keyword variable-length (**kwargs)
def info(**details):
    for k,v in details.items():
        print(f'{k}: {v}')
info(name='Ravi', city='Hyderabad')

```

5.4 Return Multiple Values

```

def calculate(a, b):
    return a+b, a-b, a*b    # Returns a tuple

result = calculate(10, 3)
print(result)    # (13, 7, 30)

# Unpack directly
add, sub, mul = calculate(10, 3)
print(add, sub, mul)    # 13 7 30

```

5.5 Lambda Functions

A lambda is a small, anonymous (nameless) function defined in a single line. Used for short operations.



SYNTAX

lambda parameters: expression

```

square = lambda x: x ** 2
print(square(5))    # 25

# With multiple parameters

```

```

add = lambda a, b: a + b
print(add(3, 7))           # 10

# Used with filter(), map(), reduce()
nums = [1,2,3,4,5,6]
evens = list(filter(lambda x: x%2==0, nums)) # [2,4,6]
squares = list(map(lambda x: x**2, nums))    # [1,4,9,16,25,36]

from functools import reduce
product = reduce(lambda x,y: x*y, nums) # 720

```

5.6 Recursion

A recursive function is one that calls itself. Every recursive function must have: (1) a base case — the condition to stop, and (2) a recursive case — calling itself with a smaller problem.

```

# Factorial: 5! = 5 × 4 × 3 × 2 × 1 = 120
def factorial(n):
    if n == 0 or n == 1: # Base case
        return 1
    return n * factorial(n-1) # Recursive call

print(factorial(5)) # 120

# Fibonacci: 0,1,1,2,3,5,8,13...
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)

print(fib(8)) # 21

```

⚠ CAUTION

Every recursive function MUST have a base case. Without it, the function will call itself infinitely and cause a RecursionError (stack overflow).

5.7 Practice Questions — Functions & Recursion

Q1. Write a function that takes a list of numbers and returns the maximum value without using max().

A1. Iterate through the list keeping track of the largest seen:

```

def find_max(lst):
    maximum = lst[0]
    for num in lst:
        if num > maximum:
            maximum = num
    return maximum
print(find_max([3,8,1,9,4])) # 9

```

Q2. Write a recursive function to calculate the sum of natural numbers up to n.

A2. Base case: sum of 0 = 0. Recursive: n + sum(n-1).

```

def sum_n(n):
    if n == 0:

```

```

        return 0
    return n + sum_n(n-1)
print(sum_n(5)) # 15

```

Q3. What is the difference between *args and **kwargs?

A3. *args collects POSITIONAL extra arguments into a TUPLE: def f(*args) — call as f(1,2,3). **kwargs collects KEYWORD extra arguments into a DICTIONARY: def f(**kwargs) — call as f(name='Ravi', age=25). Both allow functions to accept any number of arguments.

Module 6: Lists, Tuples, Sets & Dictionaries

6.1 Lists

A list is an ordered, mutable collection. Items can be of different types. Defined with square brackets [].

```

fruits = ['apple', 'mango', 'banana', 'apple']

# Access by index (0-based; -1 = last)
print(fruits[0])      # apple
print(fruits[-1])     # apple (last)

# Slicing: list[start:stop:step]
print(fruits[1:3])    # ['mango', 'banana']

# Common methods
fruits.append('grape') # Add to end
fruits.insert(1, 'kiwi') # Insert at index
fruits.remove('apple') # Remove first match
fruits.pop()           # Remove & return last
fruits.pop(0)          # Remove & return at index
fruits.sort()          # Sort ascending
fruits.reverse()       # Reverse in place
fruits.count('mango')  # Count occurrences
fruits.index('banana') # Find index
len(fruits)            # Length
fruits.clear()         # Remove all items

```

6.2 Tuples

A tuple is an ordered, IMMUTABLE collection. Once created, elements cannot be added or removed. Defined with parentheses ().

```

coords = (10, 20, 30)
print(coords[0])      # 10
print(coords[-1])     # 30
print(coords[0:2])    # (10, 20)

# Tuple unpacking
x, y, z = coords
print(x, y, z)        # 10 20 30

```

```
# Single element tuple (comma is required!)
single = (42,)          # NOT (42) which is just int

# Tuple methods
coords.count(10)        # Count occurrences
coords.index(20)        # Find index

# Convert between list and tuple
lst = list(coords)
tpl = tuple(lst)
```

**NOTE**

Tuples are faster than lists and use less memory. Use tuples for data that should not change — like coordinates, RGB values, or database records.

6.3 Sets

A set is an UNORDERED collection with NO DUPLICATES. Items are not indexed. Defined with curly braces {} or set().

```
s = {1, 2, 3, 4, 4, 3}
print(s)          # {1, 2, 3, 4} - duplicates removed

# Set methods
s.add(5)           # Add an element
s.remove(2)        # Remove (raises error if missing)
s.discard(99)      # Remove (no error if missing)
s.pop()            # Remove random element

# Set Operations (like math sets)
a = {1,2,3,4}
b = {3,4,5,6}
print(a | b)       # Union:      {1,2,3,4,5,6}
print(a & b)       # Intersection: {3,4}
print(a - b)       # Difference:  {1,2}
print(a ^ b)       # Symmetric diff: {1,2,5,6}
```

6.4 Dictionaries

A dictionary stores data as KEY : VALUE pairs. Keys must be unique and immutable. Values can be any type. Defined with curly braces {}.

```
student = {
    'name': 'Ravi Kumar',
    'age': 22,
    'course': 'CS',
    'marks': [85, 90, 78]
}

# Access values
print(student['name'])    # Ravi Kumar
print(student.get('age', 0)) # 22 (safe access)
```

```
# Add / Update
student['city'] = 'Hyderabad' # Add new key
student['age'] = 23           # Update existing

# Delete
del student['city']
student.pop('marks')

# Iteration
for key in student:           # Iterate keys
    print(key, ': ', student[key])

for k,v in student.items():   # Iterate key-value pairs
    print(k, v)

# Dictionary methods
student.keys()                # All keys
student.values()              # All values
student.items()               # All key-value pairs
'name' in student             # True - key exists?
student.update({'age':24})    # Update multiple at once
```

6.5 Comparison Table

Feature	List	Tuple	Set	Dictionary
Syntax	[1,2,3]	(1,2,3)	{1,2,3}	{'k':1}
Ordered	Yes	Yes	No	Yes (3.7+)
Mutable	Yes	No	Yes	Yes
Duplicates	Yes	Yes	No	Keys: No
Indexing	Yes	Yes	No	By key

6.6 Practice Questions — Collections

Q1. Write a program to remove duplicate elements from a list.

A1. Convert list to set (removes duplicates), then back to list:

```
nums = [1,2,3,2,4,3,5,1]
unique = list(set(nums))
print(unique)    # [1,2,3,4,5] (order may vary)
```

Q2. Given a dictionary of student names and marks, print only students who scored above 80.

A2. Iterate over dictionary items and filter by value:

```
students = {'Ravi':85,'Priya':72,'Amit':91,'Sunita':68}
for name, marks in students.items():
    if marks > 80:
        print(name, ': ', marks)
```

Q3. What is the difference between list.remove() and list.pop()?

A3. `remove(value)` removes the FIRST occurrence of a specific VALUE from the list. `pop(index)` removes and RETURNS the element at a specific INDEX (default: last element). Use `remove()` when you know the value; use `pop()` when you know the position or want the removed item back.

Module 7: Strings & File Handling

7.1 Strings

A string is an immutable sequence of characters enclosed in single, double, or triple quotes. Strings support indexing, slicing, and a rich set of built-in methods.

```
s = 'Hello, Python!'

# Indexing and Slicing
print(s[0])      # H
print(s[-1])     # !
print(s[0:5])    # Hello
print(s[7:])     # Python!
print(s[:5])     # Hello
print(s[::2])    # Hlo yhn (every 2nd char)
print(s[::-1])   # !nohtyP ,olleH (reverse)
```

String Methods

```
s = 'Hello World'
s.upper()        # 'HELLO WORLD'
s.lower()        # 'hello world'
s.title()        # 'Hello World'
s.strip()        # Remove leading/trailing whitespace
s.lstrip()       # Remove left whitespace
s.rstrip()       # Remove right whitespace
s.replace('World','Python') # 'Hello Python'
s.split(' ')     # ['Hello','World']
','.join(['a','b','c']) # 'a,b,c'
s.find('World')  # 6 (index where found, -1 if not)
s.count('l')     # 3
s.startswith('He') # True
s.endswith('ld') # True
s.isdigit()     # False
s.isalpha()     # False (has space)
len(s)          # 11
```

String Formatting

```
name = 'Ravi'; age = 22

# f-string (Python 3.6+ - RECOMMENDED)
print(f'Name: {name}, Age: {age}')

# format() method
```

```
print('Name: {}, Age: {}'.format(name, age))

# % formatting (old style)
print('Name: %s, Age: %d' % (name, age))
```

Raw Strings

```
# Normal string - backslash is escape character
path = 'C:\\Users\\Ravi' # Need double backslash

# Raw string - backslash treated literally
path = r'C:\Users\Ravi' # Much cleaner
```

7.2 File Handling

Python provides built-in functions to create, read, write, and close files. Files must be opened before use and closed after.

Opening Files

 SYNTAX	file = open('filename', 'mode')	
Mode	Name	Description
'r'	Read	Open for reading (default). Error if file doesn't exist.
'w'	Write	Open for writing. Creates file; OVERWRITES existing content.
'a'	Append	Open for writing. Creates file; ADDS to existing content.
'x'	Create	Creates new file. Error if file already exists.
'r+'	Read+Write	Open for both reading and writing.
'b'	Binary	Add to mode: 'rb', 'wb' — for images/audio/video.

```
# Writing to a file
file = open('notes.txt', 'w')
file.write('Hello, Python!\n')
file.write('Learning file handling.')
file.close()

# Reading from a file
file = open('notes.txt', 'r')
content = file.read() # Read entire file
print(content)
file.close()

# Read line by line
file = open('notes.txt', 'r')
for line in file:
    print(line.strip()) # strip() removes \n
file.close()

# BEST PRACTICE: use 'with' - auto-closes the file
```

```

with open('notes.txt', 'r') as f:
    content = f.read()
    print(content)
# File is automatically closed after the with block

# Appending to a file
with open('notes.txt', 'a') as f:
    f.write('\nNew line added!')

```

**TIP**

Always use the 'with' statement (context manager) for file operations. It automatically closes the file even if an exception occurs — preventing data loss and resource leaks.

7.3 Practice Questions — Strings & File Handling

Q1. Write a program to count the number of vowels in a string.

A1. Iterate through each character and check if it's a vowel:

```

s = input('Enter a string: ')
vowels = 'aeiouAEIOU'
count = sum(1 for ch in s if ch in vowels)
print('Vowel count:', count)

```

Q2. Reverse a string without using the reverse() function.

A2. Use string slicing with step -1:

```

s = 'Python'
reversed_s = s[::-1]
print(reversed_s) # nohtyP

```

Q3. Write a program to write 5 student names to a file and read them back.

A3. Use write mode to write, then read mode to read:

```

names = ['Ravi', 'Priya', 'Amit', 'Sunita', 'Kiran']
with open('students.txt', 'w') as f:
    for name in names:
        f.write(name + '\n')
with open('students.txt', 'r') as f:
    print(f.read())

```